Esquemas Algorítmicos

- Conocer un conjunto de técnicas de resolución de familias de problemas.

- Dado un problema concreto: caracterizar convenientemente el problema, valorar y elegir la técnica más adecuada.

El esquema voraz: Introducción

- Es uno de los esquemas más simples y al mismo tiempo de los más utilizados.
- Típicamente se emplea para resolver **problemas de optimización**:
  - Existe una entrada de tamaño *n* que son los **candidatos** a formar parte de la solución.
  - Existe un subconjunto de esos *n* candidatos que satisface ciertas restricciones: se llama **solución factible**.
  - Hay que obtener la solución factible que maximice o minimice una cierta **función objetivo**: se llama **solución óptima**.

Esquema voraz

El **esquema voraz** procede por pasos:

- inicialmente el conjunto de candidatos escogidos está vacío;
- en cada paso, se intenta añadir al conjunto de los escogidos "el mejor" de los no escogidos (sin pensar en el futuro), utilizando una **función de selección** basada en algún criterio de optimización (puede ser o no ser la función objetivo);
- tras cada paso, hay que ver si el seleccionado es **factible:**
  - si el conjunto no se puede completar, se rechaza el último candidato elegido **y no se vuelve a considerar en el futuro**;
  - si se completa, se incorpora al conjunto de escogidos y **permanece siempre en él, (función objetivo)**;
- los dos pasos anteriores, se pueden realizar juntos, es decir, al escoger un candidato ya se comprueba que sea factible o se determina que no hay candidatos.
- tras cada incorporación se comprueba si el conjunto resultante es una solución (**función de solución**);
- el algoritmo termina cuando se obtiene una solución, si se encuentra solución siempre es óptima.

Esquema voraz

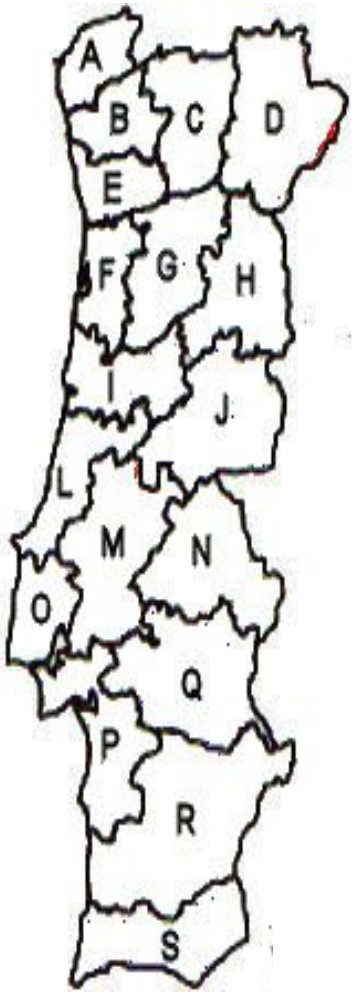
- Esquema genérico:

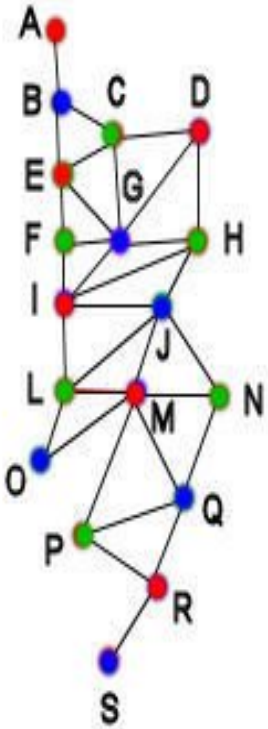
```
Conjunto voraz(C:conjunto)
{
    {C es el conjunto de candidatos}
    S←Φ {Construimos la solución en el conjunto S}

    mientras ( C != Φ y no solución( s ) )
            x ← seleccionar ( C )
            C ← C \ {x}
            Si factible ( S U {x} ) S ← S U {x}

si solución(S) devolver S
    sino devolver "no hay soluciones"
}
```

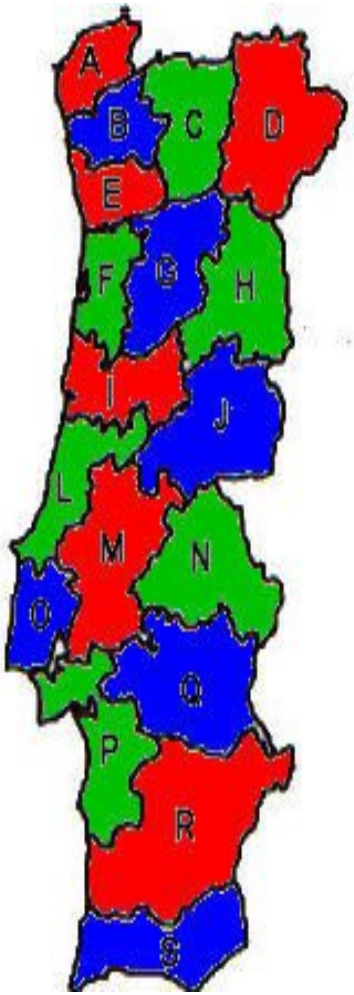
Ejemplo de Algoritmo voraz: Número cromático

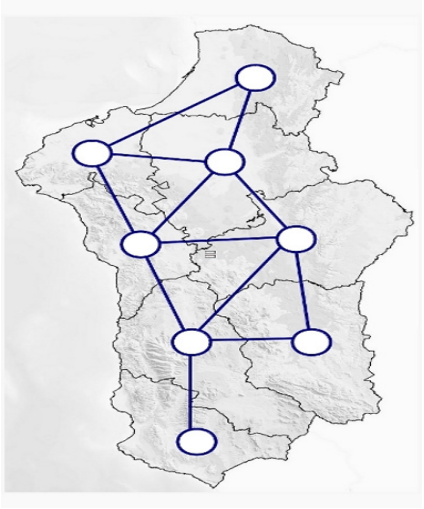
- El **algoritmo de número cromático** indica el número de colores mínimo necesario para colorear los vértices de un grafo plano. El *teorema de los cuatro colores* justifica que el número cromático de un grafo plano es menor o igual que cuatro.

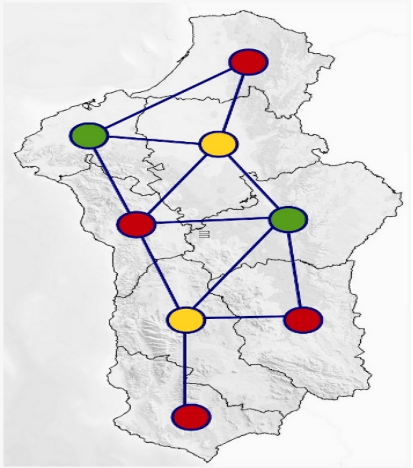
- El objetivo de este ejercicio consiste en, dado un grafo plano que representa un mapa político, indicar cómo deben colorearse cada uno de los vértices de dicho grafo, **empleando el menor número posible de colores** (según el teorema citado anteriormente como máximo serán cuatro).


A
B
C
D
E
F
G
H
I
J
L
M
N
O
Q
P
R
S


A
B
C
D
E
G
F
H
I
J
L
M
N
O
Q
P
R
S


A
B
C
D
E
F
G
H
I
J
L
M
N
O
Q
P
R
S





Ejemplo de Algoritmo voraz: Número cromático

- Siguiendo el esquema voraz:
  1. Los candidatos son los vértices del grafo.
  2. La **función de selección** toma un vértice cualquiera de los no visitados.
  3. Un vértice será **factible** si todavía no ha sido visitado. En este caso la función de selección ya comprueba si es factible.
  4. La **función objetivo** elegir un color para el vértice seleccionado, de tal forma que no sea igual a ninguno del de sus adyacentes. Objetivo colorear todos los vértices del grafo utilizando el menor número posible de colores de tal forma que dos vértices adyacentes no tengan el mismo color.
  5. La **función de solución** comprueba si ya se han visitado todos los vértices.

Ejemplo de Algoritmo voraz: Número cromático

```
función colorear(Grafo g, String [] c): colección de clave-valor
      (String)
{
    porVisitar  = V
    mapaColores vacío

    mientras porVisitar ≠ ∅ hacer
       v = vértice a colorear
       color = seleccionar color para "v" de tal forma que sea
       distinto del de sus adyacentes

        mapaColores.añadir(v,color)

    fin mientras

devolver mapaColores
}
```

```
Conjunto voraz(C:conjunto)
{

    {C es el conjunto de candidatos}
    S←Φ {Construimos la solución en el
    conjunto S}
    mientras ( C != Φ y no solución( s ) )
       x ← seleccionar ( C )
       C ← C \ {x}


       Si factible ( S U {x} ) S ← S U {x}


si solución(S) devolver S
sino devolver "no hay soluciones"
}
```

Ejemplo de Algoritmo voraz: Problema del viajante

- Consideremos un mapa de carreteras, con dos tipos de componentes: las ciudades (*nodos*) y las carreteras que las unen. Cada tramo de carreteras (*arco*) está señalado con su longitud. “Un viajante debe recorrer una serie de ciudades interconectadas entre sí, de manera que recorra todas ellas con el menor número de kilómetros posible”.

Ejemplo de Algoritmo voraz: Algoritmo del viajante

- Siguiendo el esquema voraz:
  1. Los candidatos son los vértices del grafo.
  2. Un vértice será **factible** si todavía no ha sido visitado.
  3. La **función de selección** toma el arco cuya distancia desde el vértice actual es la menor a todos los vértices que quedan por visitar (del conjunto de candidatos).
  4. La **función objetivo** obtener la distancia mínima desde el vértice origen al resto de los vértices, quedando el conjunto de candidatos vacío.
  5. La **función de solución** comprueba si ya se han visitado todos los vértices.

Ejemplo de Algoritmo voraz: Problema del viajante

```
función viajante (Grafo g, Vertice v): grafo unidirecional
{
   porVisitar  = V
   gSolucion = grafo vacio
   ciudadActual = v
   porVisitar.eliminar(ciudadActual);

  mientras porVisitar ≠ ∅ hacer
      u = Arco cuyo vertice Destino ∈ porVisitar y cuya
      distancia a ciudad actual es la mínima

      porVisitar.eliminar(u.getDestino);
      gSolucion.añadirArco(u)
      ciudadActual = u.getDestino;
  fin mientras

  devolver gSolucion
}
```

```
Conjunto voraz(C:conjunto)
{
    {C es el conjunto de candidatos}
    S←Φ {Construimos la solución en el
    conjunto S}

    mientras ( C != Φ y no solución( s ) )
      x ← seleccionar ( C )
      C ← C \ {x}

      Si factible ( S U {x} ) S ← S U {x}

si solución(S) devolver S
sino devolver "no hay soluciones"
}
```

Ejemplo de Algoritmo voraz: Algoritmo de Prim

- Consideremos un mapa de carreteras, con dos tipos de componentes: las ciudades (*nodos*) y las carreteras que las unen. Cada tramo de carreteras (*arco*) está señalado con su longitud. Se desea implantar un tendido eléctrico siguiendo los trazos de las carreteras de manera que conecte todas las ciudades y que la cantidad de tendido eléctrico utilizada sea la mínima.

Ejemplo de Algoritmo voraz: Algoritmo de Prim

- Siguiendo el esquema voraz:

  1. Los candidatos son los vértices del grafo.
  2. Un vértice será **factible** si todavía no ha sido visitado.
  3. La **función de selección,** toma el arco cuya distancia entre uno de los vértices visitados y otro de los vértices que quedan por visitar (del conjunto de candidatos) es la menor.
  4. La **función objetivo** conectar todos los vértices entre sí con la menor distancia posible.
  5. La **función de solución** comprueba si ya se han visitado todos los vértices.

Ejemplo de Algoritmo voraz: Algoritmo de Prim

```
función prim (Grafo g, Vertice v): grafo unidireccional
{
   porVisitar  = V
   gSolucion = grafo vacio
   visitados = v
   porVisitar.eliminar(v)

   mientras porVisitar ≠ ∅ hacer
      u = Arco cuyo vértice destino ∈  porVisitar y
      vértice origen ∈ visitados, cuya distancia es la
      mínima
      porVisitar.eliminar(u.getDestino)
      visitados.añadir(u.getDestino)
      gSolucion.añadirArco(u)
   fin mientras
devolver gSolucion
}
```

```
Conjunto voraz(C:conjunto)
{
    {C es el conjunto de candidatos}
    S←Φ {Construimos la solución en el
    conjunto S}


mientras ( C != Φ y no solución( s ) )
        x ← seleccionar ( C )


        C ← C \ {x}
        Si factible ( S U {x} ) S ← S U {x}

si solución(S) devolver S
sino devolver "no hay soluciones"
}
```

Ejemplo de Algoritmo voraz: Algoritmo de Dijkstra

- **Algoritmo de Dijkstra**, también llamado **algoritmo de caminos mínimos**, es un algoritmo para la determinación de la distancia más corta dado un vértice origen al resto de vértices en un grafo dirigido y con pesos en cada arista.

- La idea subyacente en este algoritmo consiste en ir explorando todos los caminos más cortos que parten del vértice origen y que llevan a todos los demás vértices; cuando se obtiene el camino más corto desde el vértice origen, al resto de vértices que componen el grafo, el algoritmo se detiene.

Ejemplo de Algoritmo voraz: Algoritmo de Dijkstra

- Siguiendo el esquema voraz:
  1. Los candidatos son los vértices del grafo.
  2. Un vértice será **factible** si todavía no ha sido visitado.
  3. La **función de selección** toma el vértice cuya distancia al vértice origen es la menor de todos los vértices que quedan por visitar (del conjunto de candidatos).
  4. La **función objetivo** modifica la distancia del vértice adyacente al seleccionado si la distancia del vértice seleccionado+ la distancia del arco entre el vértice seleccionado y el adyacente, es menor que la distancia del vértice origen al vértice adyacente. Objetivo obtener la distancia mínima al vértice origen.
  5. La **función de solución** comprueba si ya se han visitado todos los vértices.

Ejemplo de Algoritmo voraz: Algoritmo de Dijkstra

```
función dijkstra(Grafo g, Vertice v): colección de clave-valor
      (enteros)
{
    porVisitar  = V
    mapa vacío (clave los vértices, valor  la distancia)
    distancia[v] = 0 y distancia[u] = ∞ ∀ u ∈ porVisitar / u ≠ v

    mientras porVisitar ≠ ∅ hacer
      u = vertice de porVisitar cuya distancia a v es la mínima
      porVisitar.eliminar(u);

      para cada arco cuyo vértice origen sea u y vértice destino
      sea w tal que w ∈ porVisitar hacer
         si distancia[u] + w((u, w)) < distancia[w] entonces
                distancia[w] = distancia[u] + w((u, w))
         fin si
      fin para
   fin mientras
devolver distancia
}
```

```
Conjunto voraz(C:conjunto)
{

    {C es el conjunto de candidatos}
    S←Φ {Construimos la solución en el
    conjunto S}
    mientras ( C != Φ y no solución( s ) )
      x ← seleccionar ( C )
      C ← C \ {x}

      Si factible ( S U {x} ) S ← S U {x}

si solución(S) devolver S
sino devolver "no hay soluciones"
}
```